

1) Define Merkle tree

A **Merkle tree** is a binary hash tree built from all transaction hashes in a block.

- First, hash each transaction individually.
- Then pair adjacent hashes and hash each pair.
- Repeat this until only **one final hash** remains.
- That final top hash is the **Merkle root**.

This Merkle root is stored in the **block header** and acts as a compact summary of all transactions in that block.

A simple definition you can write:

A Merkle tree is a tree of hashes where leaf nodes are transaction hashes and internal nodes are hashes of child pairs, ending in one top hash called the Merkle root.

2) Define Merkle proof

A **Merkle proof** is the minimum set of sibling hashes needed to prove that one transaction belongs to a block.

Instead of showing the whole block, you only show:

- the target transaction hash
- the sibling hash next to it
- then the next sibling hash at the upper level
- and so on until the Merkle root is reconstructed

If the recomputed root matches the Merkle root in the block header, the transaction is proven to be in that block. This is why lightweight verification is possible.

Good exam definition:

A Merkle proof is a sequence of sibling hashes used to reconstruct the Merkle root and prove that a transaction is included in a block.

3) Compute a Merkle root from given transaction hashes

This is a very common numerical/simulation-style question.

Suppose sir gives:

- $H1 = \text{Hash}(\text{Tx1})$
- $H2 = \text{Hash}(\text{Tx2})$
- $H3 = \text{Hash}(\text{Tx3})$
- $H4 = \text{Hash}(\text{Tx4})$

Then you do:

- $H12 = \text{Hash}(H1 \parallel H2)$
- $H34 = \text{Hash}(H3 \parallel H4)$
- $\text{Root} = \text{Hash}(H12 \parallel H34)$

So the process is:

1. Hash each transaction
2. Pair them
3. Hash each pair
4. Keep repeating until one hash remains

If there are more transactions, same idea continues upward. The slides show exactly this Tx1, Tx2, Tx3, Tx4 style tree.

4) Prove one transaction belongs to a block

Sir may ask something like:

Prove Tx2 belongs to the block.

Then you should explain:

If we want to prove **Tx2** is included:

- start with $\text{Hash}(\text{Tx2})$
- take sibling hash $\text{Hash}(\text{Tx1})$
- compute $\text{Hash}(1,2)$
- then take the next required sibling hash from the upper level, for example $\text{Hash}(3,4)$
- compute the top hash
- if the result equals the block's Merkle root, then Tx2 belongs to that block

So you do **not** need all transactions. You only need the hashes along the path from that transaction to the root. That is the Merkle proof.

5) Compare $O(n)$ vs $O(\log n)$

This is a very important conceptual question.

Without a Merkle tree:

- to prove a transaction is in a block, you may need to inspect **all transactions**
- that is **$O(n)$** work

With a Merkle tree:

- you only need the hashes on one path from leaf to root
- the number of hashes grows with tree height
- that is **$O(\log n)$**

Meaning:

- if block has thousands of transactions, proof is still small
- verification is much faster and more scalable

The slides directly mention that Merkle proof needs only about **$\log_2(N)$ hashes**, not the whole block.

Easy wording:

$O(n)$ means work grows linearly with number of transactions. $O(\log n)$ means work grows much more slowly, because only one branch of the tree is checked.

6) Explain why Merkle root changes if one transaction changes

Because hashes are extremely sensitive to input changes.

If even **one transaction changes**:

- its leaf hash changes
- then its parent hash changes
- then grandparent hash changes
- and this effect keeps propagating upward
- so the final Merkle root changes too

That is why Merkle trees are great for tamper detection. Even a tiny change in one transaction makes the root different, which makes the block integrity check fail. The slides say changing even one transaction alters the Merkle root and invalidates the block.

Very likely exam question formats

Sir can ask in these forms:

Conceptual

- What is a Merkle tree?
- What is a Merkle proof?
- Why are Merkle trees useful in blockchain?
- Why does changing one transaction change the Merkle root?

Numerical / simulation

- Given 4 or 8 transaction hashes, compute the Merkle root.
- Given a transaction hash and sibling hashes, verify whether it belongs to the block.
- Show the Merkle proof for Tx2.

Comparison

- Why is Merkle proof $O(\log n)$ and not $O(n)$?
 - Why is Merkle tree better than checking every transaction?
-

One compact answer sheet version

Merkle tree: A binary tree of hashes where leaf nodes are transaction hashes and each parent is the hash of its two children. The top hash is the **Merkle root**, stored in the block header.

Merkle proof: A small set of sibling hashes used to reconstruct the Merkle root for one transaction and prove that it belongs to the block.

Why useful:

- summarizes all transactions in one hash
- allows lightweight verification
- detects tampering
- proof size is small, about $O(\log n)$ instead of checking all $O(n)$ transactions

Why root changes if one transaction changes: Because changing one leaf changes all hashes above it up to the root.

Also, Merkle root is one of the six fields in the Bitcoin block header, so it is definitely important.